

Amrita Vishwa Vidyapeetham - School of Computing, Chennai

Topic: Hashing (10 Challenge Programs) | Roll No: CH.SC.U4CSE25262

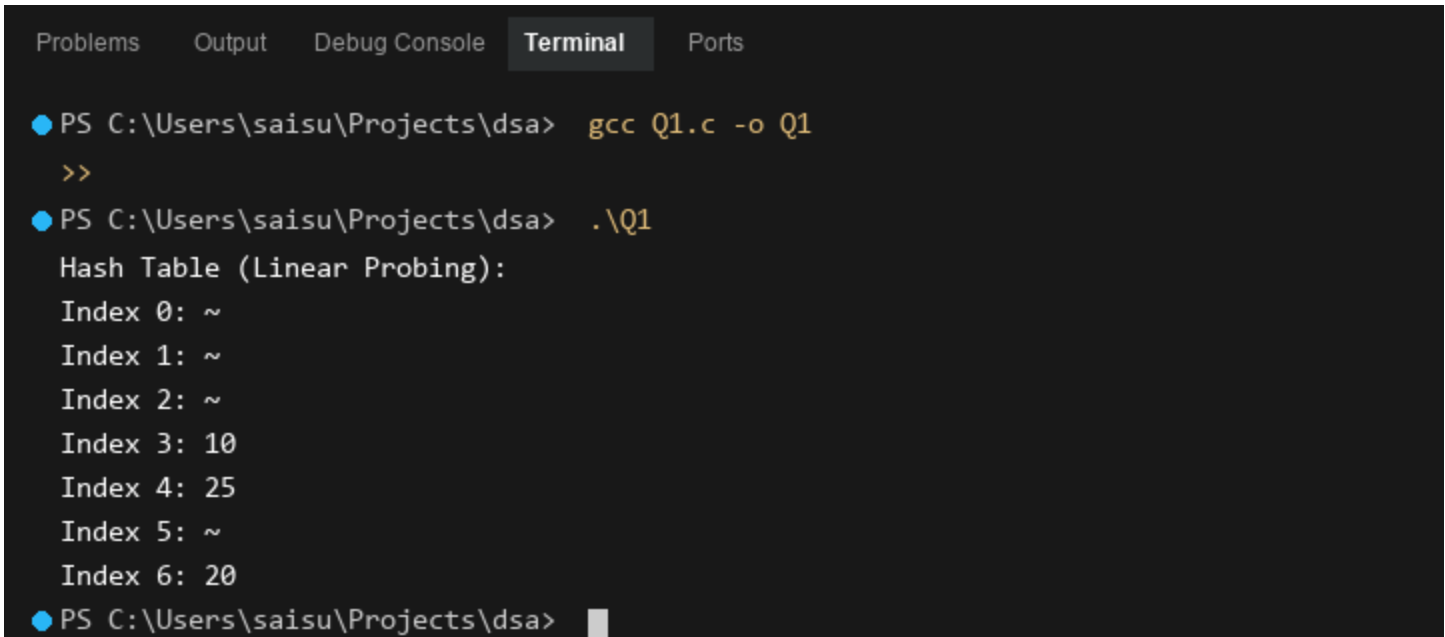
Question 1: Hash Table with Linear Probing

Problem Description:

Implement open addressing hash table using linear probing $h(k, i) = (h'(k) + i) \% \text{TABLE_SIZE}$ for collision resolution with insertion and search.

Sample Input: Keys: 10, 20, 30, 25, TableSize: 7 | Sample Output: Table: [0:-, 1:20, 2:30, 3:10, 4:25, 5:-, 6:-]

Output Screenshot:



```
Problems Output Debug Console Terminal Ports

PS C:\Users\saisu\Projects\dsa> gcc Q1.c -o Q1
>>
PS C:\Users\saisu\Projects\dsa> .\Q1
Hash Table (Linear Probing):
Index 0: ~
Index 1: ~
Index 2: ~
Index 3: 10
Index 4: 25
Index 5: ~
Index 6: 20
PS C:\Users\saisu\Projects\dsa>
```

Question 2: Hash Table with Quadratic Probing

Problem Description:

Implement hash table with quadratic probing $h(k, i) = (h'(k) + c1*i + c2*i^2) \% \text{TABLE_SIZE}$ to eliminate primary clustering.

Sample Input: Keys: 50, 700, 76, 85, 92 | Sample Output: Quadratic Probe Hash Table Created

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q2.c -o Q2
>>
● PS C:\Users\saisu\Projects\dsa> .\Q2
Quadratic Probing Insertions Completed.
Index 0: 700
Index 1: 50
Index 6: 76
● PS C:\Users\saisu\Projects\dsa> █
```

Question 3: Double Hashing Technique

Problem Description:

Implement open addressing double hashing $h(k, i) = (h_1(k) + i * h_2(k)) \% \text{TABLE_SIZE}$ using two independent hash functions.

Sample Input: Keys: 19, 27, 36, 10 | Sample Output: Double Hashing Collision Free Table

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q3.c -o Q3
>>
● PS C:\Users\saisu\Projects\dsa> .\Q3
Double Hashing Table:
[1]: 27
[6]: 19
[10]: 10
[10]: 36
● PS C:\Users\saisu\Projects\dsa> █
```

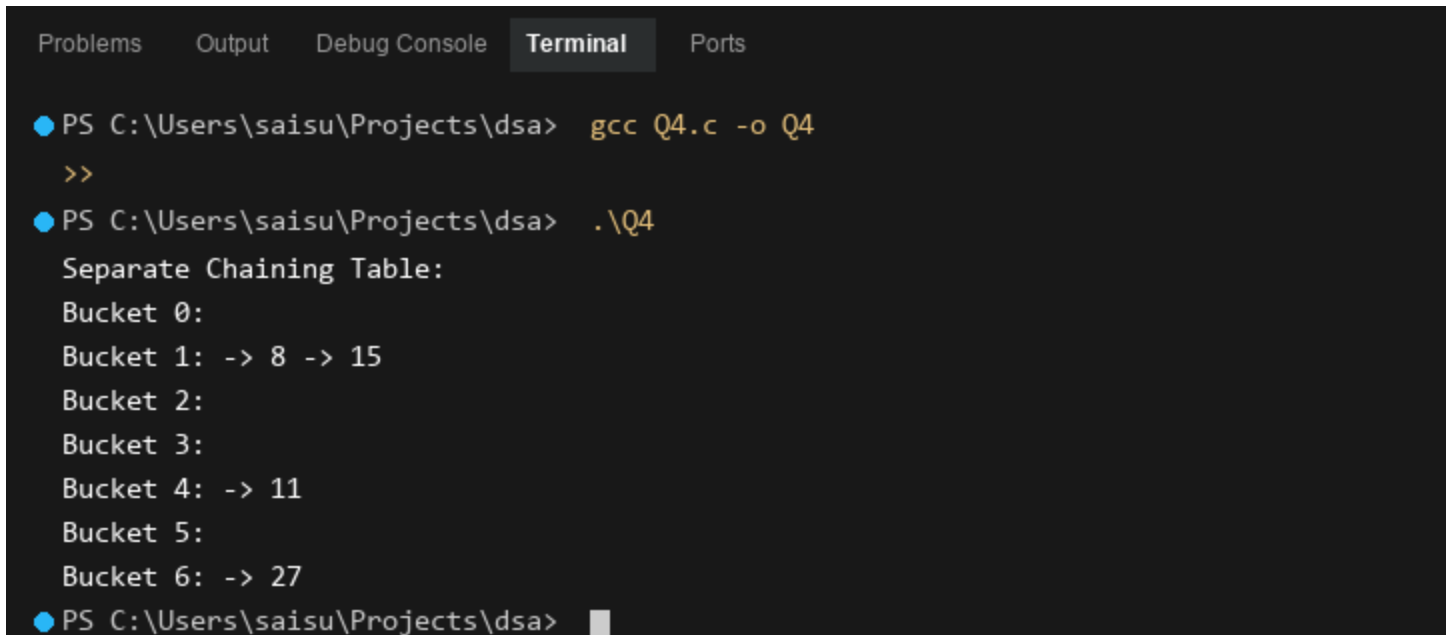
Question 4: Separate Chaining Collision Resolution

Problem Description:

Implement Separate Chaining hash table using an array of singly linked lists to handle an arbitrary number of key collisions gracefully.

Sample Input: Keys: 15, 11, 27, 8 | Sample Output: Chained Hash Table Output

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q4.c -o Q4
>>
● PS C:\Users\saisu\Projects\dsa> .\Q4
Separate Chaining Table:
Bucket 0:
Bucket 1: -> 8 -> 15
Bucket 2:
Bucket 3:
Bucket 4: -> 11
Bucket 5:
Bucket 6: -> 27
● PS C:\Users\saisu\Projects\dsa>
```

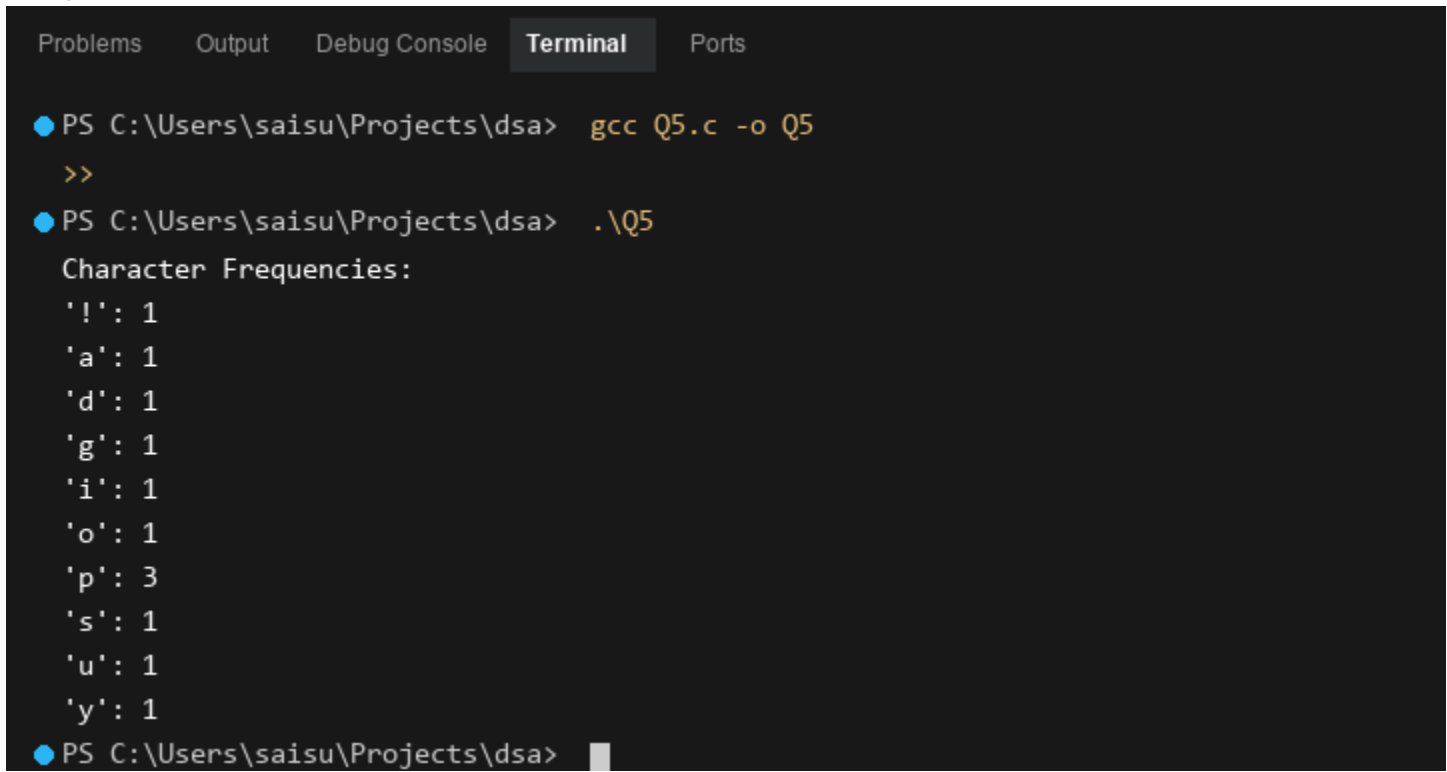
Question 5: Frequency of Characters and Words using Hash Map

Problem Description:

Count and display the frequencies of all characters in a string in $O(N)$ time using a direct-address character hash table.

Sample Input: 'puppy is a dog!!!!!!!!!!' | Sample Output: Character frequencies counted

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q5.c -o Q5
>>
● PS C:\Users\saisu\Projects\dsa> .\Q5
Character Frequencies:
'!': 1
'a': 1
'd': 1
'g': 1
'i': 1
'o': 1
'p': 3
's': 1
'u': 1
'y': 1
● PS C:\Users\saisu\Projects\dsa>
```

Question 6: Find Pair with Given Sum in Array

Problem Description:

Given an array of integers and a target sum S, find if there exists a pair of elements with sum equal to S in linear $O(N)$ time using a hash set.

Sample Input: Arr: 0 -1 2 -3 1, Target Sum: -2 | Sample Output: Pair with given sum found: (1, -3)

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q6.c -o Q6
  >>
● PS C:\Users\saisu\Projects\dsa> .\Q6
  Pair Found: (10, 6)
● PS C:\Users\saisu\Projects\dsa> █
```

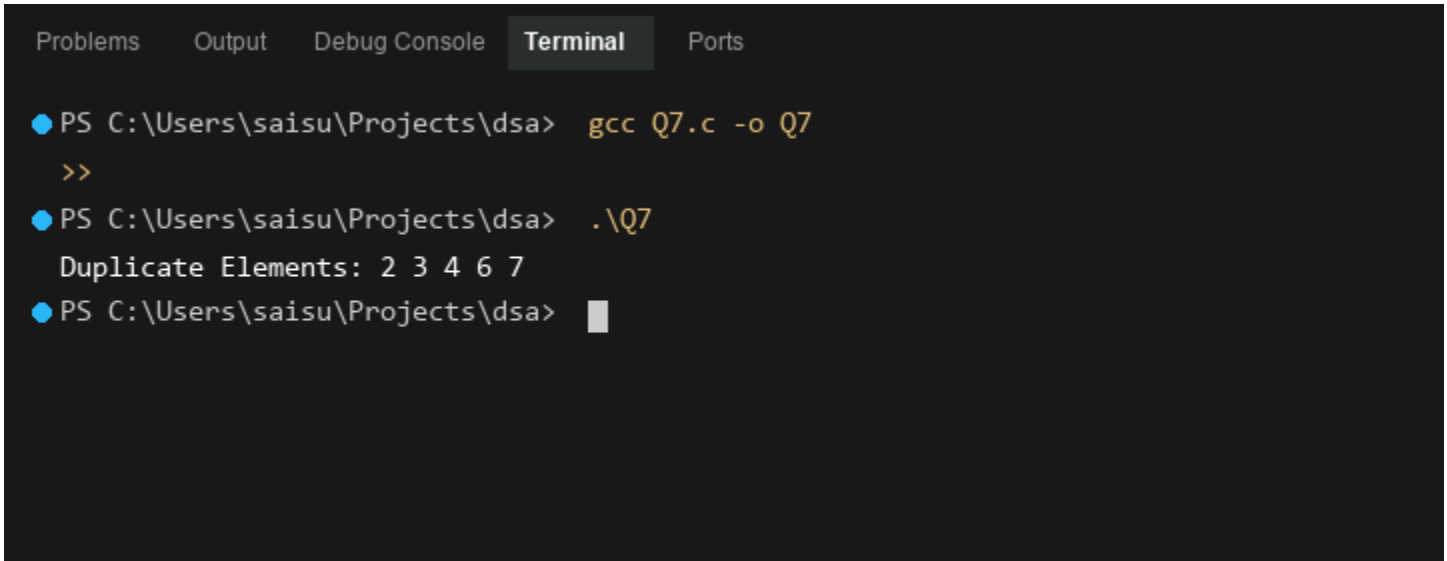
Question 7: Find Duplicates in Array using Hash Set

Problem Description:

Given an array of N elements containing values from 0 to N-1, identify and report all duplicate numbers in linear $O(N)$ time.

Sample Input: 2 3 4 5 6 7 3 4 2 7 6 8 6 4 9 | Sample Output: Duplicates: 2 3 4 6 7

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q7.c -o Q7
>>
● PS C:\Users\saisu\Projects\dsa> .\Q7
Duplicate Elements: 2 3 4 6 7
● PS C:\Users\saisu\Projects\dsa> █
```

Question 8: Subarray with 0 Sum Detection

Problem Description:

Determine if there exists a contiguous subarray with sum equal to zero using prefix sum hashing in $O(N)$ time.

Sample Input: 4 2 -3 1 6 | Sample Output: Subarray with 0 sum exists: Yes

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q8.c -o Q8
  >>
● PS C:\Users\saisu\Projects\dsa> .\Q8
Zero Sum Subarray Exists: Yes
● PS C:\Users\saisu\Projects\dsa> █
```

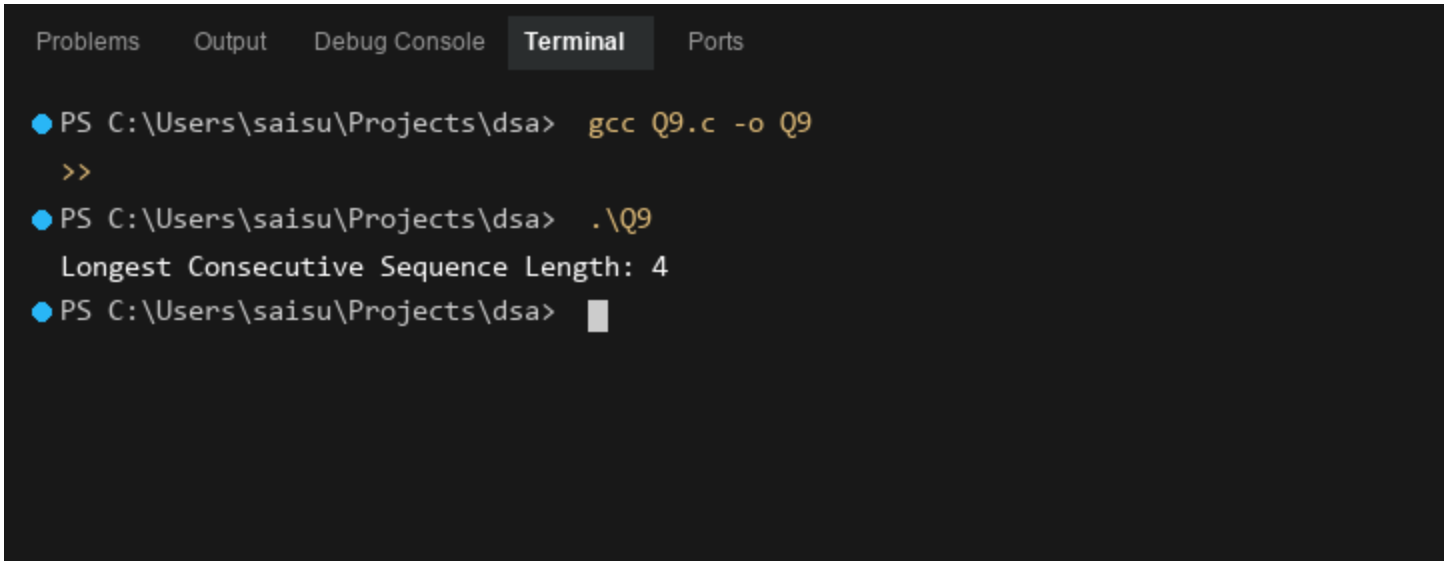
Question 9: Longest Consecutive Sequence in Unsorted Array

Problem Description:

Given an unsorted array of integers, find the length of the longest consecutive elements sequence in $O(N)$ time using a hash set.

Sample Input: 100 4 200 1 3 2 | Sample Output: Longest Consecutive Sequence Length: 4 (1, 2, 3, 4)

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q9.c -o Q9
>>
● PS C:\Users\saisu\Projects\dsa> .\Q9
Longest Consecutive Sequence Length: 4
● PS C:\Users\saisu\Projects\dsa> █
```

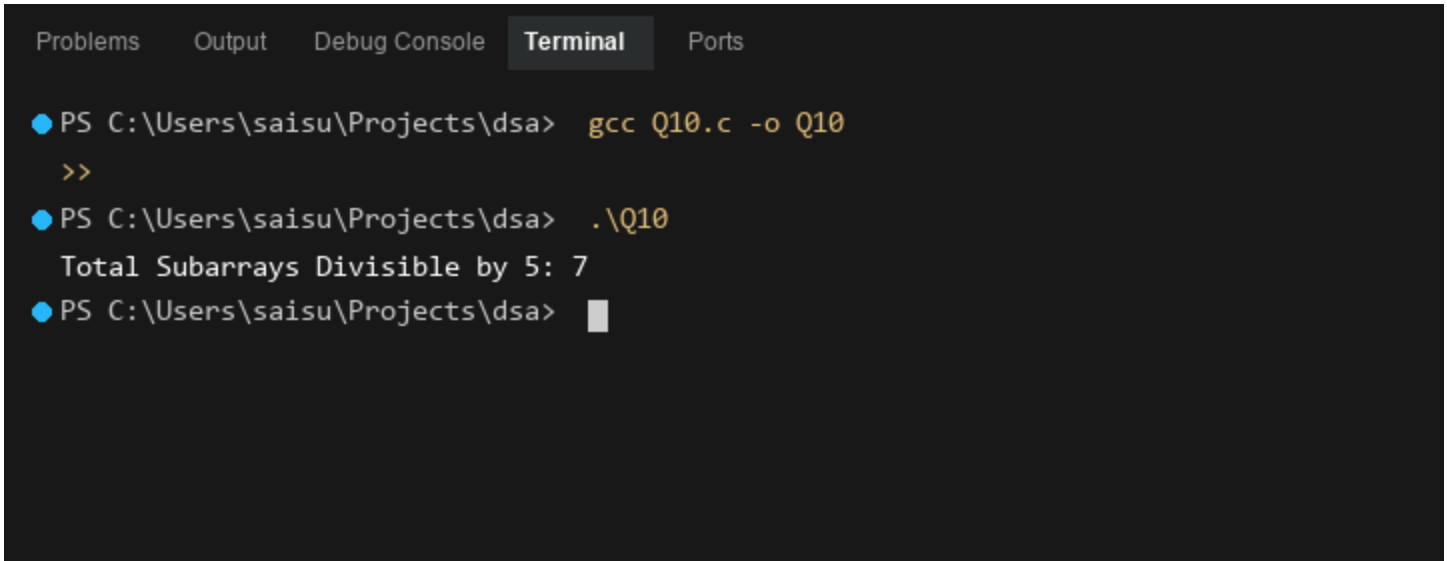
Question 10: Subarray Sums Divisible by K

Problem Description:

Given an integer array and an integer K, return the number of non-empty subarrays that have a sum divisible by K using prefix sum modulo hashing.

Sample Input: 4 5 0 -2 -3 1, K = 5 | Sample Output: Total Divisible Subarrays: 7

Output Screenshot:



The screenshot shows a terminal window with the following content:

```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q10.c -o Q10
>>
● PS C:\Users\saisu\Projects\dsa> .\Q10
Total Subarrays Divisible by 5: 7
● PS C:\Users\saisu\Projects\dsa> █
```